Flutter Lab — Refactor to Cubit

ITI Online Course — Duration: 3 hours — Same app as before, new architecture

The Idea

No new features today. Take your **exact same app** from the last lab (Login, Products, Edit, Favorites, Logout) and refactor its state management from `setState` to **Cubit**, one piece at a time. By the end, the app must look and behave identically to a user — the entire difference is internal: no more manually passing data between screens, no more duplicated favorite-toggling logic between Products and Favorites.

> Do not delete your working setState version until the Cubit version fully replaces it screen by screen — refactor incrementally, testing after each step, not all at once at the end.

Step 1 — Setup + AuthCubit (35 min)

- Add `flutter_bloc` to `pubspec.yaml`.
- Create `AuthState` (Initial, Loading, LoggedIn, Error, LoggedOut) and `AuthCubit` wrapping your existing `ApiService.login()` and `logout()` methods — don't rewrite the API calls themselves, just move the state around them into the Cubit.

Step 2 — Refactor LoginScreen (35 min)

- Convert `LoginScreen` to `StatelessWidget`.
- Replace the manual loading/error state variables with `BlocConsumer<AuthCubit, AuthState>` — `listener` handles navigation on `AuthLoggedIn`, `builder` handles the loading spinner and error text.
- Test: wrong password shows the error, correct login navigates to Products — exactly like before, just no `setState` left in the file.

Step 3 — ProductsCubit (Products + Favorites Together) (55 min)

- Create `ProductsState` (Loading, Loaded — carrying BOTH the product list and the current favorite IDs, Error) and `ProductsCubit`.
- Move `getProducts()`, `updateProduct()`, `deleteProduct()` calls into Cubit methods that `emit()` the right state.
- Move the SharedPreferences favorites logic into the Cubit's `toggleFavorite()` — it must update both the persisted list AND emit a new `ProductsLoaded` state in one method.

Step 4 — Refactor ProductsScreen, EditProductScreen, FavoritesScreen (40 min)

- Convert all three screens to `StatelessWidget`.
- `ProductsScreen` and `FavoritesScreen` both read from the SAME `BlocBuilder<ProductsCubit, ProductsState>` — `FavoritesScreen` just filters by the favorite IDs already in that state.
- `EditProductScreen` can stay simpler — it just calls `context.read<ProductsCubit>().updateProduct(...)` on save instead of returning data through `Navigator.pop`.

Step 5 — Wire Up main.dart + Prove Nothing Broke (35 min)

- Wrap the app in `MultiBlocProvider` with both Cubits, injecting the same `ApiService` instance into both.

- Full regression pass: login (wrong then correct), browse products, edit one, favorite 2-3, open Favorites and confirm they match, un-favorite one from Favorites screen directly and confirm Products reflects it immediately, delete a product, log out and confirm Back doesn't return to Products.

Hint: if toggling a favorite on the Favorites screen itself doesn't instantly update, it's a strong sign that screen isn't reading from the shared ProductsCubit — a very common mistake at this stage.

Mandatory Rules

- Zero `setState()` calls should remain anywhere in the refactored screens.
- Zero manual data-passing between Products and Favorites — both must read the same Cubit.
- The app's visible behavior must be indistinguishable from last week's version to an end user.
- Every screen touched must be converted to `StatelessWidget`.

Time Allocation (suggested)

Step	Time
1 — Setup + AuthCubit	35 min
2 — Refactor LoginScreen	35 min
3 — ProductsCubit (products + favorites)	55 min
4 — Refactor Products/Edit/Favorites screens	40 min
5 — Wire main.dart + full regression test	35 min

Note: the real success metric today isn't new visible features — it's less code, no manual data passing, and instant sync between Products and Favorites. If a student can't tell the difference from outside the app but the code is visibly simpler, they've understood the point of the lab.